DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Thunking

CMSC 124 Activity 13

Prepared by: Rene Andre Bedonia Jocsing

Published: October 26, 2026 **Deadline:** October 28, 2026

The same call can mean copied values, caller aliases, delayed accesses, or output cells that don't have values yet. Today you'll run one procedure under five parameter-passing methods. The procedure stays fixed, but what reaches it changes.

The programs here are pseudocode. They're written to be read. Restart each trace from its stated initial values. Arrays use zero-based indexing, so `[10, 20, 30]` has valid indices 0, 1, and 2. An invalid read or array access stops that trace before the assignment completes.

Laboratory Activity 5 turns this into code. A function value in your language carries its environment. The final sections connect environments to delayed access and local storage.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 22 points. Twelve points reward traces, and ten reward explanations.

The Five Contracts

Consider this pseudocode:

```
blend(p, q):
    p = p * 2
    q = q + p

u = 3
v = 4
blend(u, v)
```

The **formal parameters** `p` and `q` are placeholders declared by `blend`. The **actual parameters** `u` and `v` are the caller expressions supplied by this call. A **cell** is a storage location that holds a value.

Copy-in copies an actual value into a formal cell at entry. **Copy-out** copies a final formal value back to its fixed actual target at return. An **alias** is another access to the same caller cell. A **delayed access** reevaluates the actual expression whenever the formal is read or written.

All five contracts below answer the same question: What does the formal parameter stand for while the body runs?

1. **Pass-by-value** answers a private copy, so assignments to these integer formals leave the caller cells alone.
2. **Pass-by-result** answers an empty cell the callee has to fill, so the caller learns the answer at return and contributed nothing at entry.
3. **Pass-by-value-result** answers a copy that gets shipped home, so the caller learns the answer at return but supplied the starting value.

4. **Pass-by-reference** answers the caller's own cell, so the caller sees each write the instant it happens.
5. **Pass-by-name** answers the caller's expression, still unevaluated, so the caller sees each write immediately and the expression may land on a different cell each time it runs.

Use these exact rules:

Method	Entry	Access in the callee	Return
Value	Copy each actual value into a local formal cell	Read and write the local cell	Discard local cells
Result	Create an uninitialized local formal cell	The callee may assign but must not read before assignment	Copy each final formal value to its actual target
Value-result	Copy actual values into local formal cells	Read and write the local cells	Copy final values back left to right
Reference	Bind each formal to the caller cell selected at call time	Read and write the caller cell directly	No copy-out step
Name	Bind each formal to a delayed access for its actual expression	Reevaluate that access on every read and write	No copy-out step

For pass by result and value-result, actual targets are fixed when the call begins. Today's copy-out order is left to right so collision outcomes are determinate.

Worked Example for Checkpoint 1

Suggested time: 33 minutes, including the worked example.

Use a body that assigns `p` before reading it, so all five methods are valid:

```
set(p, q):
    p = 7
    q = p + 1

a = 5
b = 6
set(a, b)
```

Start each method with `a = 5` and `b = 6`. In the tables, "uninitialized" means no value is available to read. It doesn't mean zero.

Value

The call copies 5 and 6 into two private cells. Watch which cells each assignment changes.

Step	Formal cells	Caller cells
Entry: copy in	`p = 5`, `q = 6`	`a = 5`, `b = 6`
`p = 7`	`p = 7`, `q = 6`	`a = 5`, `b = 6`
`q = p + 1`: read local `p = 7`, store 8	`p = 7`, `q = 8`	`a = 5`, `b = 6`
Return: discard both locals	no formal cells remain	`a = 5`, `b = 6`

Result

The caller supplies destinations, but no starting values. Entry fixes `a` and `b` as the copy-out targets. The first assignment supplies the value that the second assignment needs.

Step	Formal cells	Caller cells
Entry: no copy-in	`p`, `q` uninitialized	`a = 5`, `b = 6`
`p = 7`	`p = 7`, `q` uninitialized	`a = 5`, `b = 6`
`q = p + 1`: read local `p = 7`, store 8	`p = 7`, `q = 8`	`a = 5`, `b = 6`
Return: copy `p` to `a`	`p = 7`, `q = 8`	`a = 7`, `b = 6`
Return: copy `q` to `b`, discard locals	no formal cells remain	`a = 7`, `b = 8`

Value-result

Entry fixes the same destinations and also copies their values into local cells. The body still can't change the caller through those locals. Copy-out does that later.

Step	Formal cells	Caller cells
Entry: copy in	`p = 5`, `q = 6`	`a = 5`, `b = 6`
`p = 7`	`p = 7`, `q = 6`	`a = 5`, `b = 6`
`q = p + 1`: read local `p = 7`, store 8	`p = 7`, `q = 8`	`a = 5`, `b = 6`
Return: copy `p` to `a`	`p = 7`, `q = 8`	`a = 7`, `b = 6`
Return: copy `q` to `b`, discard locals	no formal cells remain	`a = 7`, `b = 8`

Result and value-result finish alike here because `set` overwrites both formals before their starting values could matter. Their entry states differ.

Reference

There are no private value cells for `p` and `q`. Entry binds `p` to the caller's `a` cell and `q` to its `b` cell. Those bindings stay fixed throughout the call.

Step	Access through formals	Caller cells
Entry	`p` aliases `a`, `q` aliases `b`	`a = 5`, `b = 6`
`p = 7`	write directly to `a`	`a = 7`, `b = 6`
`q = p + 1`	read `a = 7`, write 8 to `b`	`a = 7`, `b = 8`
Return	no copy-out	`a = 7`, `b = 8`

Name

Entry binds delayed accesses for the expressions `a` and `b`. Each formal read or write evaluates its actual access again. These simple variables select the same cells every time.

Step	Access through formals	Caller cells
Entry	retain accesses for `a` and `b`, copy no values	`a = 5, b = 6`
`p = 7`	evaluate access `a`, write 7	`a = 7, b = 6`
`q = p + 1`	evaluate access `a`, read 7, evaluate access `b`, write 8	`a = 7, b = 8`
Return	no copy-out	`a = 7, b = 8`

Reference fixes cells at entry. Name retains expressions that select cells on each access. The distinction doesn't change this call's result, but it will matter when an array index changes.

The five figures below are one call under five contracts. The procedure body is identical in all five. What changes is what the formals stand for, and when the caller sees a write.

Five parameter methods, five cell relationships

The procedure body is fixed. Each snapshot changes what the formal parameters denote and when caller cells are updated.

Copy values into local cells

Pass by value changes local `p` and `q` to 7 and 8, then discards them. Caller cells remain 5 and 6.

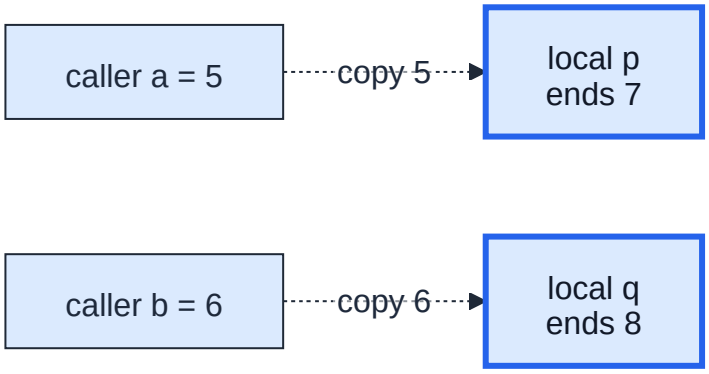


Figure 1: Copy values into local cells

Copy results out of uninitialized cells

Pass by result starts with empty formals. The first assignment defines `p`; the second may then read it. Copy-out stores 7 and 8 in the caller.

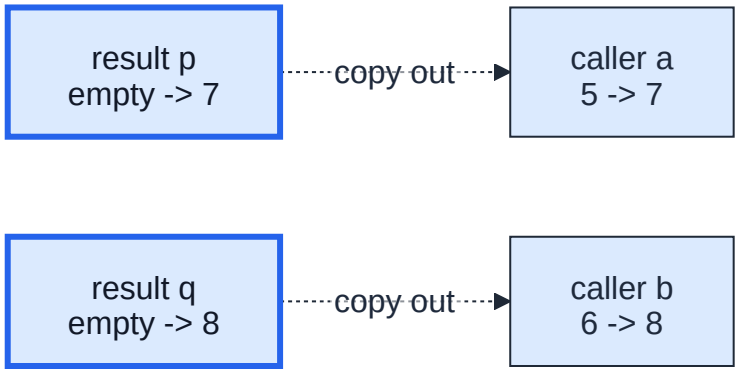


Figure 2: Copy results out of uninitialized cells

Copy in, then copy out

Pass by value-result begins with local copies 5 and 6, changes them to 7 and 8, then copies both results back.

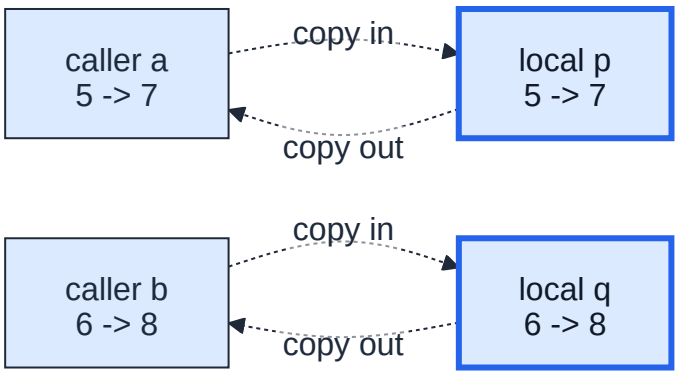


Figure 3: Copy in, then copy out

Alias caller cells directly

Pass by reference binds `p` to `a` and `q` to `b`. Assignments in the body immediately update the caller.

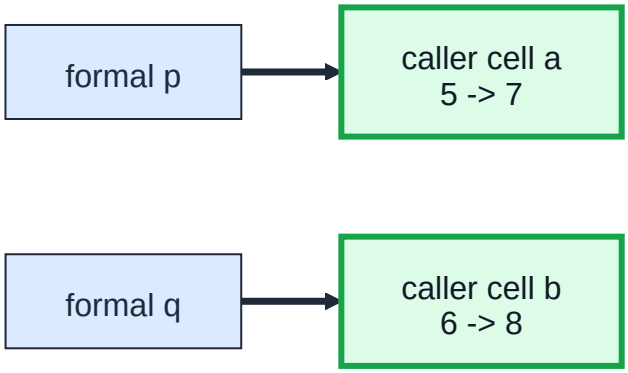


Figure 4: Alias caller cells directly

Reevaluate delayed accesses

Pass by name reevaluates the access for `a` or `b` on every formal read and write. Simple variable actuals still reach the same caller cells.

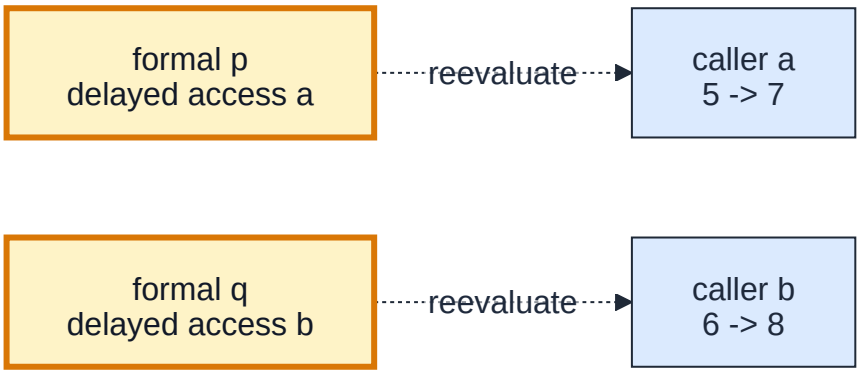


Figure 5: Reevaluate delayed accesses

Checkpoint 1: Run All Five Contracts (10 Points)

Analyze `blend(u, v)` with initial `u = 3` and `v = 4`:

```
blend(p, q):
    p = p * 2
    q = q + p
```

Under value, the formals are local copies. Under result, they begin uninitialized and copy out at return. Under value-result, they copy in and copy out left to right. Under reference, they alias the caller cells selected at entry. Under the final contract, every formal access reevaluates its delayed actual expression.

1. (Construction: 6 points) Make one row per method. Record what `p` and `q` denote after entry, their state after each assignment, any copy-out, and the final caller state. If a contract makes the body invalid, stop at the first invalid operation and identify it.

2. (Explanation: 4 points) Answer all three for `blend(u, v)`. Use your trace to support each explanation.

a. Under pass-by-value, which cells do the assignments to `p` and `q` update? Explain why `u` and `v` keep their initial values after the call returns. (1 point)
b. Under pass-by-result, which read in `p = p * 2` is invalid? Explain why the assignment cannot give `p` its first value. (1 point)
c. Compare pass-by-value-result, pass-by-reference, and pass-by-name. For each method, state when the caller cells `u` and `v` change. Then explain why all three produce the same final caller values for these two distinct variable actuals. (2 points)

Collide the Actual Parameters

Restart with `u = 3` and call `blend(u, u)`. This is an **actual-parameter collision** because both actual parameters designate the same caller variable.

Worked Example for Checkpoint 2

Suggested time: 16 minutes, including the worked example.

Use `set` from Checkpoint 1, but supply `a` for both actual parameters:

```
set(p, q):
```

```
    p = 7
    q = p + 1

a = 5
set(a, a)
```

Restart with `a = 5` for each method below. The caller has one cell, but the two formals don't necessarily share it during the body.

Value-result

Entry copies the same value into two separate local cells. It also fixes the same caller cell `a` as the copy-out destination for both formals.

Step	Local formals	Caller cell
Entry: copy in twice	`p = 5`, `q = 5`	`a = 5`
`p = 7`	`p = 7`, `q = 5`	`a = 5`
`q = p + 1`: read local `p = 7`, store 8	`p = 7`, `q = 8`	`a = 5`
Return: copy `p` to `a`	`p = 7`, `q = 8`	`a = 7`
Return: copy `q` to `a`, discard locals	no formal cells remain	`a = 8`

Changing local `p` doesn't change local `q`. The collision occurs at return, when both results go to the same destination. The second copy-out overwrites the first. Reverse that order and `q` first writes 8 to `a`, then `p` writes 7. The final value would be 7.

Reference

Entry makes both `p` and `q` aliases of `a`. A write through either formal changes the one cell that both formals read.

Step	Access through formals	Caller cell
Entry	`p` and `q` both alias `a`	`a = 5`
`p = 7`	write 7 to `a`; both formals now read 7	`a = 7`
`q = p + 1`	read 7 from `a` through `p`, write 8 to `a` through `q`	`a = 8`
Return	no copy-out	`a = 8`

Name

Entry retains the actual expression `a` for each formal. Every read or write evaluates its delayed access again. Here, both expressions select the same caller cell every time.

Step	Delayed accesses	Caller cell
Entry	retain access `a` for `p` and access `a` for `q`	`a = 5`
`p = 7`	evaluate `p`'s access `a`, write 7	`a = 7`
`q = p + 1`	evaluate `p`'s access to read 7, then `q`'s access to write 8	`a = 8`
Return	no copy-out	`a = 8`

All three methods finish with `a = 8` under the specified copy-out order. Their intermediate states differ. After `p = 7`, value-result still has local `q = 5` and caller `a = 5`. Under reference and delayed access, a read through `q` would already see 7.

`set` doesn't read the old value of `q`, so that difference doesn't affect its final answer. The checkpoint's `blend` body does read `q` in `q = q + p`. Keep that read visible when you trace the collision.

Checkpoint 2: Make Aliasing Visible (7 Points)

Restart with `u = 3` and run this exact call and body:

```
blend(u, u)

blend(p, q):
    p = p * 2
    q = q + p
```

1. (Construction: 4 points) Trace the call under reference, value-result, and the delayed-access contract. Show both reads in `q = q + p`. For value-result, show the two local final values and each left-to-right copy-out.

2. (Explanation: 3 points) Explain why reference and delayed access share changes during the body while value-result delays the collision until copy-out. State how reversing the copy-out order would change the value-result outcome.

Transfer the Delayed Access

Worked Example for Checkpoint 3

Suggested time: 11 minutes, including the worked example.

Return to `set` for this worked example:

```
set(p, q):
    p = 7
    q = p + 1

data = [10, 20, 30]
i = 1
set(i, data[i])
```

Restart with this state for each contract. The array has three cells. Keep in mind that index 7 isn't one of them.

Reference

Entry binds `p` to `i`. It evaluates the second actual's index once, while `i = 1`, and binds `q` to `data[1]`. That binding survives changes to `i`.

Step	Access through formals	`i`	`data` (indices 0, 1, 2)
Entry	`p` aliases `i`, `q` aliases `data[1]`	1	`[10, 20, 30]`
`p = 7`	write 7 to `i`, keep `q` bound to `data[1]`	7	`[10, 20, 30]`

Step	Access through formals	`i` `data` (indices 0, 1, 2)
`q = p + 1`	read `i = 7`, compute 8, write to the fixed cell `data[1]`	7 `[10, 8, 30]`
Return	no copy-out	7 `[10, 8, 30]`

The second assignment doesn't evaluate `data[i]` again. It writes through the binding established at entry. The final state is `i = 7`, `data = [10, 8, 30]`.

Name

Entry retains the expressions `i` and `data[i]` as delayed accesses. It doesn't fix an array cell for `q`. Each use evaluates the corresponding access with the caller's current state.

Step	Delayed accesses	`i` `data` (indices 0, 1, 2)
Entry	retain access `i` for `p` and access `data[i]` for `q`	1 `[10, 20, 30]`
`p = 7`	evaluate access `i`, write 7	7 `[10, 20, 30]`
Read `p` in `q = p + 1`	evaluate access `i`, read 7, compute 8	7 `[10, 20, 30]`
Attempt to write `q`	reevaluate `data[i]` as `data[7]`, invalid access, stop	7 `[10, 20, 30]`

Computing 8 succeeds. Storing it fails. The right side reads only `p`, so the first invalid operation is the attempted write through `q`. No array element receives 8. The earlier write to `i` remains. There is no normal return. The stopped state is `i = 7`, `data = [10, 20, 30]`.

Compare the destinations below. The red box marks the failed attempt to access index 7. The array contains only indices 0, 1 and 2.

Fixed cell or reevaluated access?

After the first assignment changes i to 7, the two contracts select different destinations for the second assignment.

Reference keeps the cell selected at entry

Entry binds q to data[1]. Changing i to 7 does not change that binding. The second assignment writes 8 to data[1].

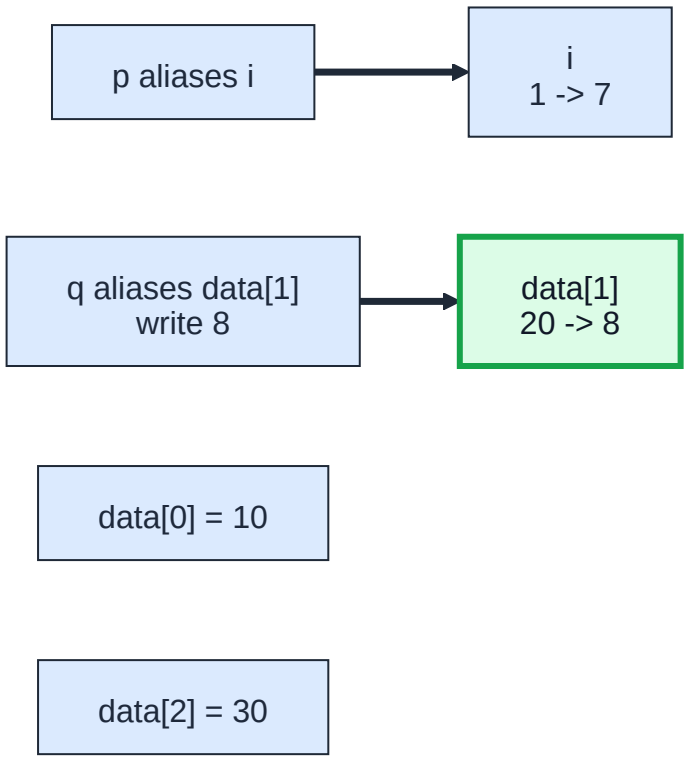


Figure 6: Reference keeps the cell selected at entry

Delayed access uses the current index

After i becomes 7, writing q reevaluates data[i] as data[7]. That cell does not exist, so the trace stops without an array write.

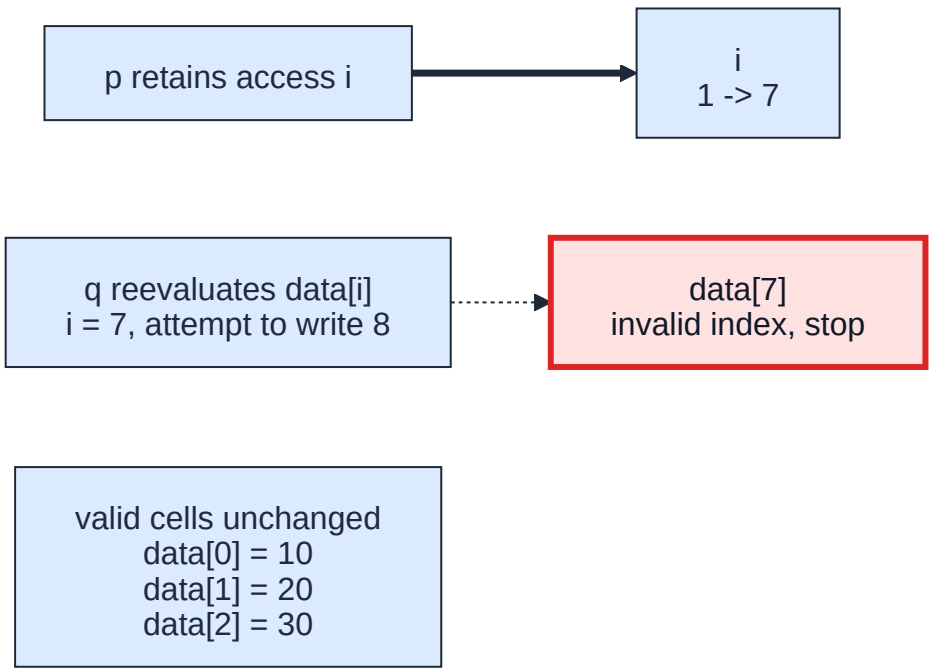


Figure 7: Delayed access uses the current index

The delayed item is an access expression. It doesn't save the original value 20 or the original destination `data[1]`. Keeping `data[1]` under both methods ignores reevaluation after `i` changes.

The section just after this checkpoint shows how a compiler builds a delayed access. Working the behavior out from the contract first is the better exercise. If you want the mechanism before you use it, it sits one section ahead.

Checkpoint 3: Let the Address Move (5 Points)

Use this fresh state and call:

```
blend(p, q):
    p = p * 2
    q = q + p

data = [10, 20, 30]
i = 1
blend(i, data[i])
```

1. (Construction: 2 points) Trace the call once under reference and once under the delayed-access contract. Record `i` and every array element after each assignment.

2. (Explanation: 3 points) Explain why reference keeps using the array cell selected at entry while delayed access selects a different cell after `i` changes. Identify the wrong turn of treating a delayed access as a delayed value.

End with both final arrays beside the access rule that selected their changed element.

How the Delayed Access Is Built

Here's one way to implement the delayed-access contract. For each assignable actual expression, build two small functions at the call site, one to read it and one to write it. A function that delays this work is called a **thunk**. Each thunk carries access to the caller's environment, so its variables refer to the caller's cells.

Return to the worked example's `set(i, data[i])`, with `i = 1` and `data = [10, 20, 30]`. The compiler builds one pair per parameter. If you've tackled the proxy design pattern before, then this should look familiar:

```
function p_read():
    return i

function p_write(newval):
    i = newval

function q_read():
    return data[i]

function q_write(newval):
    data[i] = newval
```

None of these thunk bodies runs at call entry. The callee receives the functions and their environment. When it calls a thunk, `data` and `i` refer to the caller's current cells. The thunk reads their values at that moment.

The function and the environment it retains form a **closure**. Here, that environment gives both pairs access to the same `i`. The `q` pair also has access to `data`. Changing `i` through `p_write` therefore changes the index that `q_write` will use. Creating the functions doesn't read `data[1]` or fix it as a destination.

Now translate the callee's accesses into thunk calls. A formal read calls its read thunk. A formal write calls its write thunk:

```
set(p, q):                    compiles to    set():
    p = 7                                        p_write(7)
    q = p + 1                                    q_write(p_read() + 1)
```

The thunk calls implement reevaluation. `p_write(7)` changes the caller's `i` to 7. Next, `p_read()` returns 7, so the argument to `q_write` is 8. Its body then evaluates `data[i]` with `i = 7`.

`set` never reads `q`, so it never calls `q_read`. The failure occurs inside `q_write`, when it selects the destination. Follow the retained accesses in the first figure, then the order of calls in the second:

Thunks connect the callee to caller cells

The callee receives functions that retain access to the caller's cells. Calling a function performs the delayed read or write.

Keep the functions with their environment

Both pairs use the same caller cell i. The q pair also uses data, so every array access uses the current index.

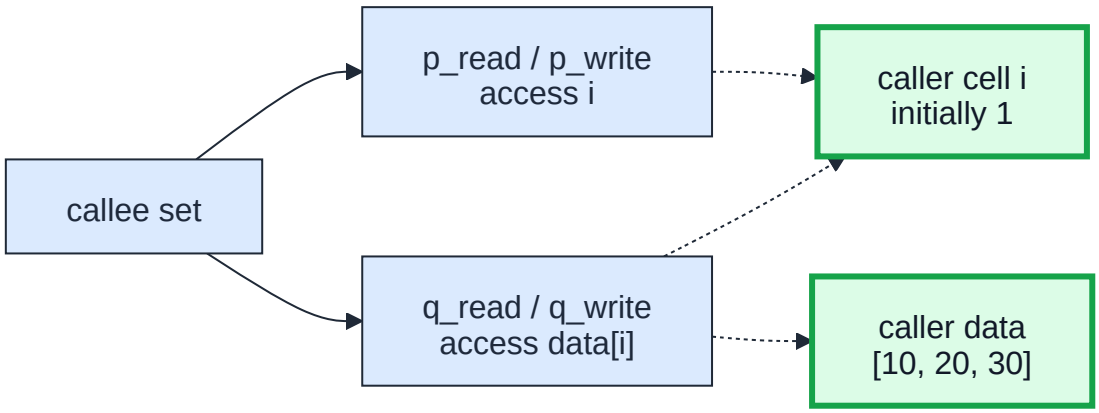


Figure 8: Keep the functions with their environment

Run the translated body

Writing p changes i before q's write thunk runs. The value 8 is ready, but selecting data[7] fails.

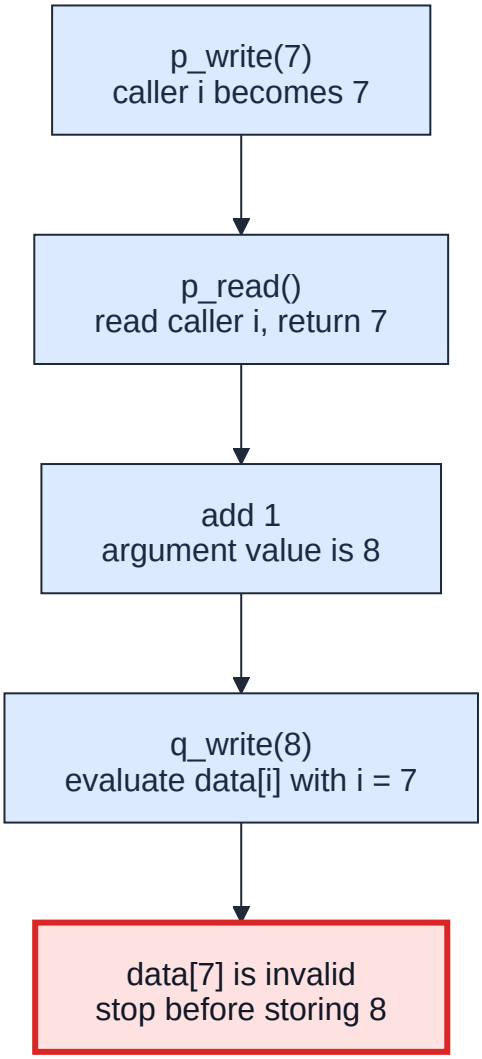


Figure 9: Run the translated body

For this three-element array, index 7 is out of bounds. The write fails. The array stays `[10, 20, 30]`. The delayed access selects its destination at write time, even when that destination is invalid.

What the Callee Keeps

The traces above ask which cell a formal accesses. Now ask how long that cell exists. Under value and value-result, the private formal cells belong to one call. A reference or thunk can instead access cells that belong to the caller. Returning from the callee doesn't discard those caller cells.

A subprogram's **local referencing environment** contains its local variables. A cell's **storage lifetime** is the period during which its storage exists. For a local to retain its value between calls, its cell must survive return. In this pseudocode, `local` creates a fresh cell per call, while `static` creates one shared cell initialized once before the first call.

```
count(n):
    local seen = 0
    static total = 0
    seen = seen + n
    total = total + n
```

`seen` is **stack dynamic**. Its cell is allocated when the call begins and released when the call returns,

so every call gets a fresh one and the previous call's value is gone. `total` is a **static local**. Its cell persists between calls, so the second call reads whatever the first call left in it.

Call `count(2)` and then `count(3)`. Keep the calls in that order:

Step	This call's `seen`	Shared `total`
Enter `count(2)`	allocate a fresh cell, initialize to 0	0
Finish both additions	2	2
Return from `count(2)`	release the cell	2
Enter `count(3)`	allocate a new cell, initialize to 0	2
Finish both additions	3	5
Return from `count(3)`	release the cell	5

The second call starts `seen` over. It doesn't start `total` over. The two snapshots show which cell is new and which one persists:

One fresh local and one persistent local

Each call creates its own seen cell. Both calls update the same total cell, which survives return.

First call, count(2)

seen starts at 0 and reaches 2 before its cell is released. The persistent total starts at 0 and retains 2 after return.

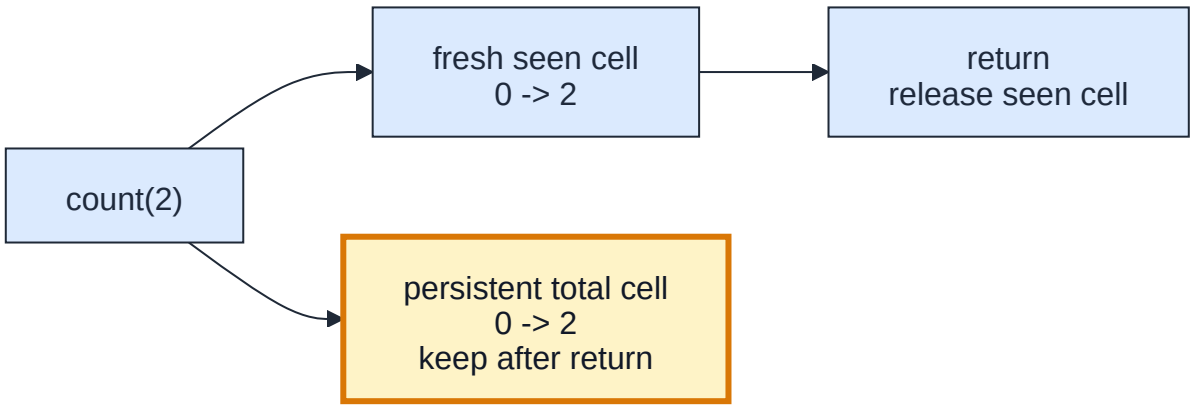


Figure 10: First call, count(2)

Second call, count(3)

A new seen cell starts at 0 and reaches 3. The same total cell starts this call at 2 and retains 5 after return.

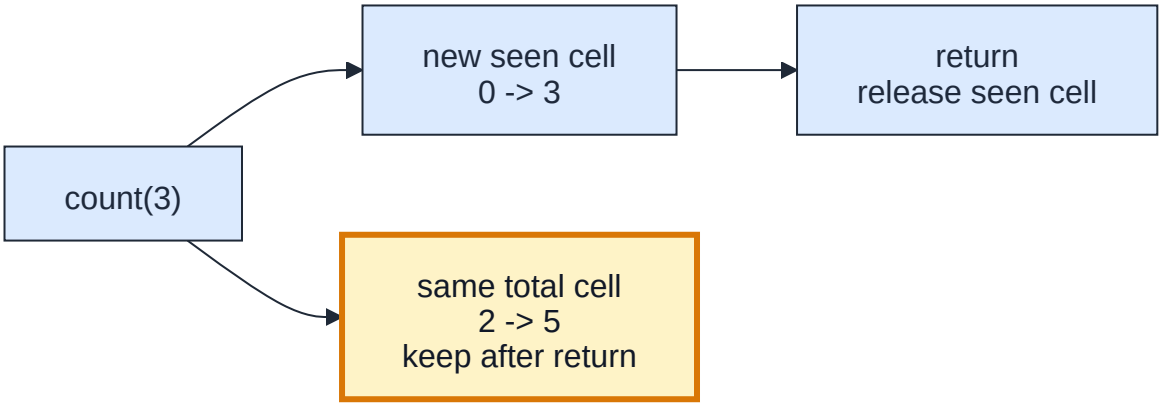


Figure 11: Second call, count(3)

The thunk example and `count` share a rule about cells. Two accesses share changes only when they reach the same cell. The thunk pairs share the caller's `i`, and both `count` calls share static `total`. Each call gets its own stack-dynamic `seen`.